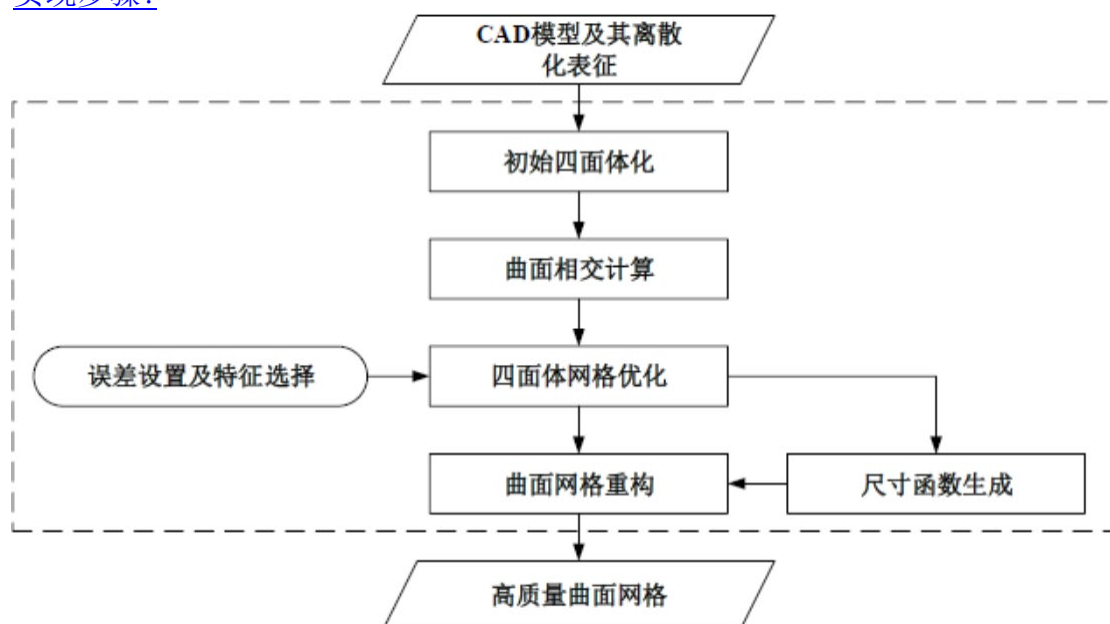
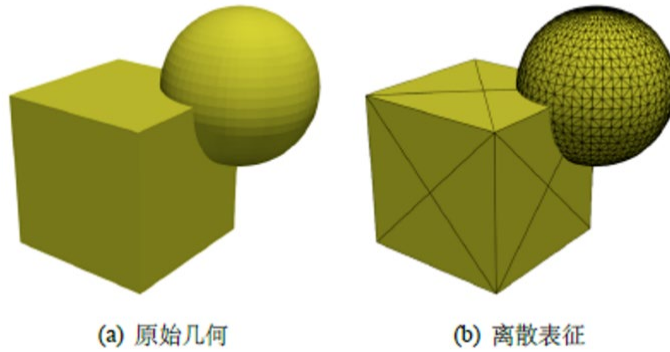


类 TAU 网格技术方案

- 方案目标:
 - 基本概念: 完成初步算法开发，能针对多腔体滤波器，采用先体网格、再面网格的算法生成初始网格，网格质量达到 HFSS 的 85%。
 - 本方案针对串行的电磁初始网格生成。
- 技术背景:
 - HFSS 的 TAU 网格针对一些特定类型网格，可以减少迭代轮数、降低网格数量；
 - TAU 网格可以处理脏几何情况。
- 技术选择:
 - 采用先体后面的网格生成流程，实现类 TAU 网格；
 - 采用曲面相交计算、四面体网格优化、曲面网格重划分等技术实现对脏几何的自动处理。
- 架构设计:
 - 基于 YunboMech，实现先体后面的网格生成流程；
 - 流程中调用各项网格功能 API。
- 实现步骤:



1. 几何转换获得初始 STL;



方案 1: 使用 OCC 生成初始 STL 网格:

读取几何到内存中

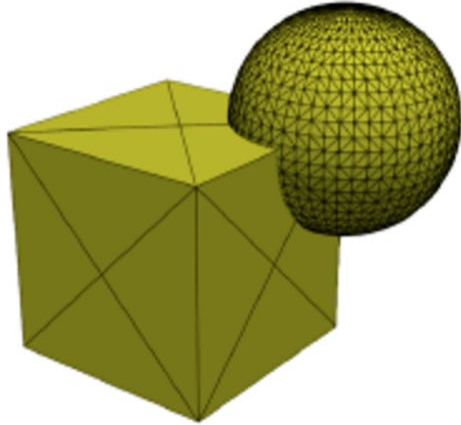
```
Tiger_API void readModel_OCC(  
const std::string& filename,  
TiGER::Geometry& geometry,  
const readModelParameters& args  
);
```

生成 STL 利用 OCC 引擎

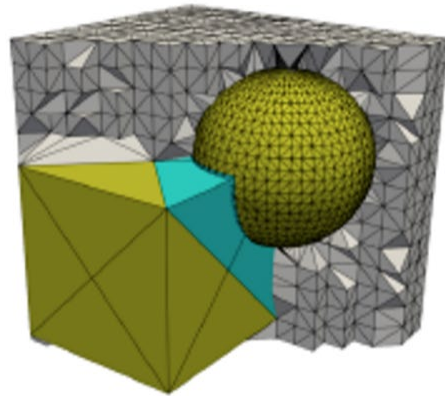
```
Tiger_API void generateStl_OCC(  
TiGER::Geometry& geometry,  
TiGER::SurfaceMesh& surfaceOut,  
TiGER::generateStlParameters& arg_stl  
);
```

方案 2: 使用 InterOP 生成初始 STL 网格:

2. 初始四面体网格生成(初始 Delaunay+边界恢复)



(b) 离散表征



(c) 初始四面体化

其中蓝色面片是因为相交而恢复失败的网格面片

方案 1: 复用 YunboFluid 四面体生成算法

方案 2: 使用浙大 Tiger 库 API:

Tiger:

保形的边界恢复, 其中过程包含四面体化、约束边界恢复、求交算法等核心步骤

@param input_mesh 输入的表面网格, 可能包含相交、重叠、狭缝、交缠等几何错误

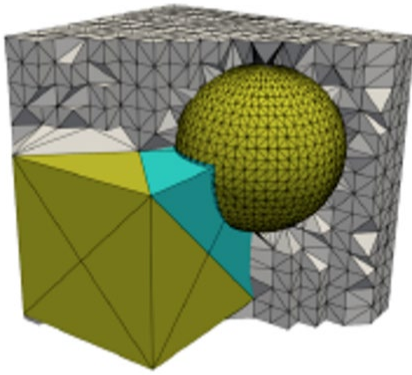
@param tet_mesh 输出的保形的体网格

@param connector 辅助信息, 例如特征边、局部容差等

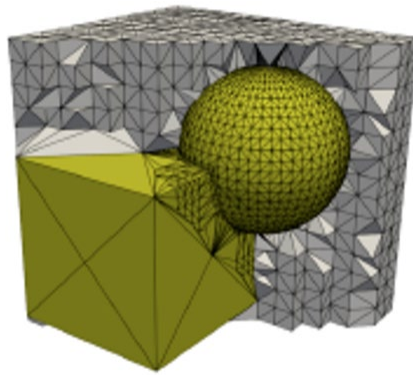
@param args 控制参数列表

```
Void tahedralization(  
Mesh &input mesh,  
Mesh &tet mesh,  
conformingConnector &connector,  
const Args &args  
);
```

3. 三角面片求交（要支持三角形面网格属性设置）



(c) 初始四面体化



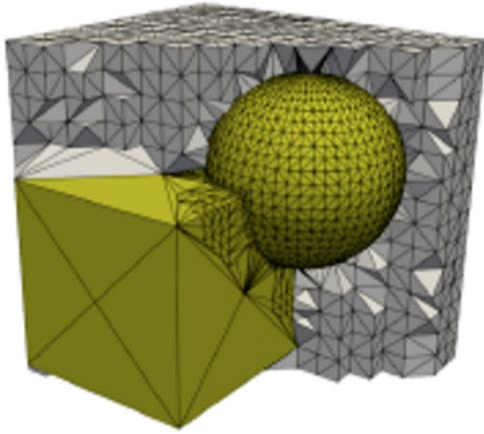
(d) 相交计算

通过求交，将所有边界恢复失败的面片重新得到，保证网格的保形

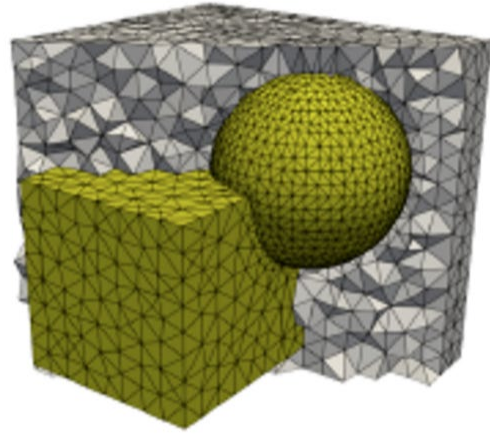
Tiger:

没有单独接口，目前是所有三角形一起求交点，然后在连接拓扑

4. 四面体网格优化(带容差的优化, 基于包络的优化)



(d) 相交计算



(e) 四面体优化

基于四面体优化的质量度量, 优化四面体网格, 在这个过程中, 要保证对边界的修改在容差范围内, 且保证网格质量的提高。

方案一:

复用 YunboFluid 包面网格生成算法

方案二:

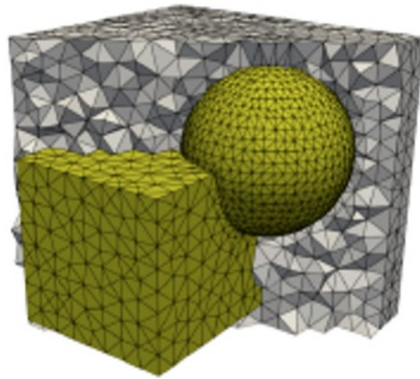
保形的, 有 Hausd 距离的网格优化, 需要辅助信息, 例如特征边、局部容差等

```
void tet_improving(  
    Mesh &tet mesh, connector &connector,  
    const Args &args,  
    Mesh &output _mesh  
);
```

方案三:

Mmg3d, tetgen

5. 抽取得到初始面网格



(e) 四面体优化

将优化得到四面体网格中，属于原始 stl 贴合部分的面网格抽离出来，即上图中的黄色部分。

直接写在网格优化中，未作分离。

6. 重构尺寸函数

方案一：

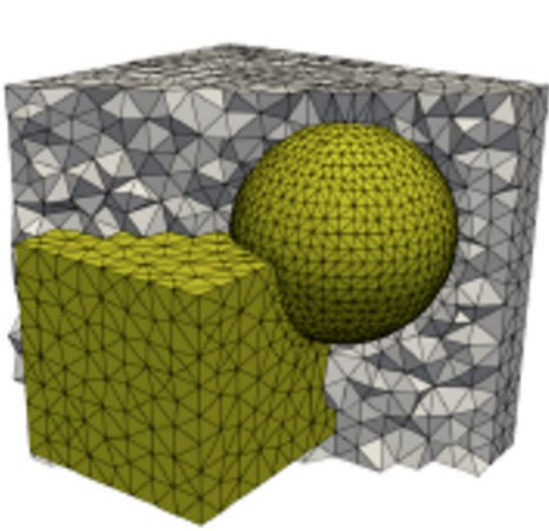
复用 YunboMech 尺寸函数

方案二：

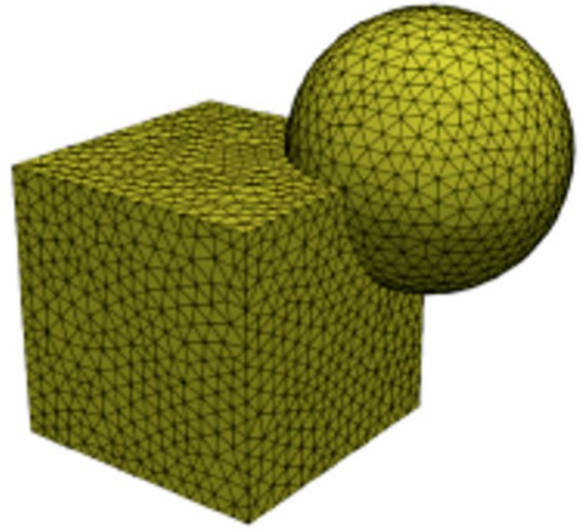
（有基于离散尺寸、基于连续的尺寸）

```
Tiger_API void setSizeField(  
    const SurfaceMesh& SurfaceMesh,  
    const BackGroundParameters& args,  
    SizingManager& manager  
);
```

7. 面网格重构获得高质量网格（尺寸函数控制）



(e) 四面体优化



(f) 曲面网格重构

如果对网格质量有要求，对抽出来的面网格进行面网格重构。获得高质量的面网格

方案一：

复用 YunboMech 曲面网格重划分算法

方案二：

```
Tiger_API void triangleRemesh(  
    const SurfaceMesh &surfmesh_in,  
    const std::vector<std::vector<int>> &constrained_edge,  
    const std::vector<std::vector<int>> &conforming_edge,  
    const RemeshParameters &args,  
    std::function<double(double, double, double)> &size_function,  
    SurfaceMesh &surfmesh_out,  
    std::vector<std::vector<int>> &constrained_edge_out  
);
```

8. 四面体网格生成（基于面网格生成体网格）
将生成的面网格作为体网格的输入，生成四面体网格。

方案 1：复用 YunboMech 四面体网格生成算法

方案 2：复用 YunboFluid 四面体网格生成算法

方案 3：

```
Tiger_API void constrainDelaunay(  
    const SurfaceMesh& surface_mesh,  
    const TetrahedraParameters& args,  
    VolumeMesh& volume_mesh,  
    const SizingFunction& func  
);
```


- TAU 网格为什么可以加速一些网格类型的自适应猜想
 - TAU 网格对模型的几何特征更好的识别与自适应，更贴体。
 - TAU 网格清理非必要的细节、或许有助于加快迭代过程
- 需要的三方库(协议 LGPL)
 - Eigen
 - libIGL
 - Spdlog
 - Cll11
 - GMP(有理数计算)
 - OCC

- 开发周期:

实现先体后面的调用流程: 2 周

各项网格功能 API 方案对比: 2 周

性能优化, 达到 1230 验收标准的 70%: 2 周

性能优化, 达到 1230 验收标准: 2 周

- 方案风险:

1. 浙大的算法依据实现步骤, 但是 API 不是, 存在嵌套或多个流程只有一个 API, 无法简单的通过 API 叠加实现。
2. 类 TAU 网格为了有良好的网格质量, 其时间开销不可避免的会增加, 如何平衡时间和质量是一个问题。
3. 脏几何的情况过于复杂, 无法完全自动生成, 仍然依赖人工干预。
4. 输入的 STL 影响 TAU 网格的迭代步骤与收敛速度, 一个好的初始 STL 至关重要, 可以作为拔高算法的一步。
5. 各向异性网格是减少网格量, 加快网格收敛速度的一个重要技术, 是一个未来考虑的重要方向。